

# MINspire 2026: Day 4 Assignment

## Day 5: Decision Trees, SVMs, Ensembles & Clustering Name

### Q1. Session Summary

The topics today involved classical machine learning algorithms. Decision Trees use Gini impurity for splitting the feature space, but deep decision trees cause overfitting. Support Vector Machines identify the optimal margin hyperplane through the help of kernel functions in case of nonlinearly separable data sets, and support vectors from that hyperplane. Ensembles involve many weak learners; random forests use bagging and random feature selection for variance reduction purposes, whereas Gradient Boosting uses sequential trees to minimize residual error. Unsupervised learning algorithms include K-Means, which forms  $k$  spherical clusters via alternating assignments and update steps, and DBSCAN, which identifies arbitrary shapes and outliers based on densities.

### Q2. Overfitting Trees and How Random Forests Help

Such a tree that keeps dividing till it perfectly matches the training data is simply remembering that particular data set along with all the noise in it, instead of learning the general form of that function. In other words, this tree suffers from overfitting. This means that the accuracy during training will seem perfect but that the variance of the model is too high and will not generalize well with new data points.

A solution to this problem can be achieved using random forests, wherein several trees are generated but are decorrelated from one another. This means that each tree is generated based on different bootstrap samples of the data, and at each split of the data, only a random selection of features may be selected for generating the tree. These trees would continue to suffer from overfitting; however, due to decorrelation, these errors would cancel out once averaged in case of regression and voting in case of classification.

### Q3. Choosing $k$ for K-Means With No Prior Knowledge

Without any prior information, you use your data to estimate  $k$  by applying two diagnostic procedures that are used together across a number of candidate values of  $k$ .

The elbow technique: make an inertia vs.  $k$  graph and identify the "elbow" point, which is the point beyond which adding additional clusters will not result in a significant reduction in inertia. Inertia always goes down as  $k$  increases; the elbow is a point of diminishing returns rather than a minimum point.

The Silhouette: because the elbow technique does not work very well in practice, you check for the best value of  $k$  by calculating the mean Silhouette for every candidate  $k$ . This technique selects the value of  $k$  that gives the maximum mean Silhouette, which means that the clusters produced are compact while also being well-separated. The combination of these two techniques works much better than either one on its own. Otherwise, you could opt for DBSCAN as a density-based clustering approach.

#### **Q4. Paper Summary: "Random Forests," Leo Breiman (2001)**

The problem this paper solved: Breiman asked whether it was possible to create a classifier that would be not only very accurate but also robust (would not overfit and would be resistant to noise) through the use of ensembles made up of a large number of randomised decision trees, as well as offering internal diagnostics.

Key result: The combination of the technique known as bagging (in which each tree is trained on a bootstrap sample) and random feature selection results in a forest with its generalisation error approaching some limit, regardless of how many trees are used – i.e., adding trees will never cause overfitting. The error is bounded by two parameters, one of which is the correlation between the trees, and the lower this correlation, the smaller the error – which explains why this random feature selection was necessary. In terms of accuracy, performance and built-in diagnostics, random forests compared favorably with AdaBoost, the top algorithm at that time.

#### **Q5. Week 1 Reflection**

Support vector machines (SVMs) took the most effort out of all of them initially. I had no trouble with maximizing the margin by constructing an optimal hyperplane, but the step up to applying the kernel trick seemed confusing – using a kernel instead of dot product meant doing the calculations in higher dimensions even though you never compute those coordinates directly. The realization only came in class when we saw how RBF worked by looking at a decision boundary for it.

The loop consisting of the assignment and the updating step in the K-Means algorithm. It would go like that – select  $k$  clusters randomly, then perform the following operations iteratively – associate all the points to their nearest cluster center, and relocate each cluster center to the mean of all the associated points. We do that until our centers stabilize; this always converges but only to the nearest solution, therefore we apply this process several times.